
Editorial for the Test Contest

long, long ago

A. Sum of Numbers I

For all his prudence, the merchant Aristarkh Pankratyevich has set a simple task: observe that the answer is simply the number $n + m$. No further reasoning is required – read n and m , add them, and print the result. Under the given constraints ($1 \leq n, m \leq 1000$) the sum is certain to fit in any integer type, so overflow need not concern us. The solution runs in $O(1)$. ■

Editorial for the Test Contest

long, long ago

B. Sum of Numbers II

The solution is identical to that of problem A: the answer is $n + m$. The split into subgroups here serves a purely didactic purpose – to remind the reader that not every subgroup makes the algorithm harder; sometimes it merely narrows the range of the input while leaving the solution unchanged. A correct solution for the first subgroup is therefore automatically correct for the second as well, and writing separate code for $(1 \leq n, m \leq 500)$ would be a needless expense of ink. ■

Editorial for the Test Contest

long, long ago

C. Sum of Numbers III

For all his forgetfulness, the station keeper has set the very same task: the answer is the sum $n + m$. Full marks are awarded for passing all the tests of subgroup 1, which include the sample tests as well. The problem holds no further tricks – it suffices to read two numbers and print their sum. ■

Editorial for the Test Contest

long, long ago

D. Sum of Numbers IV

Algorithmically the problem does not differ from the previous three: the answer is $n + m$. What deserves attention is only the merged-subgroup scoring: to receive the points for subgroup 2, a solution must pass **all** the tests of both subgroup 1 and subgroup 2 – in other words, the points for subgroup 2 are not awarded on their own, but only in addition to an already-passed subgroup 1. Since the same solution works for every constraint, a correct submission receives the full hundred points at once. ■

Editorial for the Test Contest

long, long ago

E. Sum of Numbers V

Here too the answer is $n + m$, read and printed without any special contrivance. The only difference from problems A–D lies in the scoring: each of the two additional tests, beyond the sample tests, is scored independently (50 points apiece) rather than as a single subgroup. In practice this makes no difference to the solver – a partial solution is simply impossible here, since the problem is either solved in full or not solved at all. ■

Editorial for the Test Contest

long, long ago

F. Guess the Number

Cornet Sven offers a classic exercise in **binary search**. Maintain the bounds of the interval of possible values $[lo, hi]$, initially $lo = 1$, $hi = t$. At each step query the midpoint $x = \lfloor \frac{lo + hi}{2} \rfloor$:

- on the reply = – the number has been guessed; terminate at once;
- on the reply < (the number sought is less than x) – shrink the interval to $[lo, x - 1]$;
- on the reply > – shrink the interval to $[x + 1, hi]$.

The interval shrinks by at least half with every query, so the number of queries never exceeds $\lceil \log_2 t \rceil$. For $t \leq 10000$, fourteen queries suffice – comfortably within any reasonable hidden bound k . Remember to flush the output after every query, and to terminate the program the moment the reply = is received. ■

Editorial for the Test Contest

long, long ago

G. Add 1

The problem is best understood by splitting it into the roles of the clerk and the official. In the first run, the program reads a number n and prints $n + 1$ – nothing more complicated than adding one is required here. In the second run, the program receives on its input exactly the number printed by the first run (the judge itself carries this value from the first run’s standard output to the second run’s standard input – no temporary files need be created), and it must print $v - 1$, that is, the original n .

The chief difficulty of the problem is not algorithmic but organizational: one must ensure that both runs read and write through the standard streams exactly once, with no extraneous prompts or debug output that might confuse the judging system. ■